Practical - 1

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

A. Theory

1. What is URL and parts of URL?

Answer:

A URL (Uniform Resource Locator) is a unique address used to locate and access resources on the internet.

- Identifies the location of a resource online.
- Specifies the protocol (e.g., HTTP or HTTPS).
- Includes the domain name or IP address.
- May contain a path to a specific resource.
- Used to access web pages, files, images, and other online resources.

PARTS OF URL

- Scheme: Defines the protocol used to access the resource (e.g., http, https, ftp).
- Subdomain: Specifies a subdivision of the main domain (e.g., www).
- Domain: Identifies the main website name (e.g., example).
- Top-Level Domain (TLD): Indicates the domain extension (e.g., .com, .org).
- Port Number: Specifies the server port (default: 80 for HTTP, 443 for HTTPS).
- Path: Points to the resource location on the server.
- Query String Separator: The ? symbol that starts query parameters.
- Query String / Parameters: Passes data as key-value pairs using &.
- Fragment: Refers to a specific section within the resource using

https://www.geeksforgeeks.org/computer-networks/components-of-a-url/

2. How does HTML file process in web browser?

Answer:

You Open the File in a Browser
When you double-click the file or open it through a web server, a browser like Chrome, Firefox, or Safari reads the code.The Browser Parses the HTML
The browser scans your code from top to bottom. It starts with the <! DOCTYPE html> declaration, then reads the file and then browser builds the page and display the content.

https://dev.to/ofodile/how-html-works-in-the-browser-25pa

3. How will page navigation flow managed among all HTML pages?

Answer:

Each page has links to the other pages, such as Home, Courses, Assignments, Contact Us, and Profile. When a user clicks a link, the href attribute opens the required HTML page.

4. How will GitHub commits be maintained after each practical?

Answer:

After each practical, you should maintain your GitHub commits by staging your completed files, writing a clear descriptive commit message, and pushing your local updates to the remote GitHub repository.

https://github.com/git-guides/git-commit

B. Evidence of Work (Screenshots)

 STUDENT HUB

Welcome to Your Learning Portal

Home | Courses | Assignments | Contact Us | Profile

| Welcome Student! |

Access all your academic information from one place

| 📖 Courses | | 📝 Assignments | | 📂 Materials |

View Courses Check Tasks View Notes

[Open] [Open] [Open]

@ 2026 Student Hub | All Rights Reserved

C. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

D. Challenges Faced:

- Deciding the proper layout and placement of elements on each page.
- Maintaining the same design and style across all 10 pages.
- Creating a simple and easy-to-use navigation flow.
- Managing different sections like Courses, Assignments, Notes, and Profile.
- Making the design attractive while keeping it simple and user-friendly

E. Learning Outcomes:

- Understood how different pages are connected through navigation and sitemap.
- Improved knowledge of creating wireframes and arranging UI elements properly.
- Learned to maintain consistency in fonts, colours, buttons, and page layouts.
- Got practical experience in designing a multi-page StudentHub portal.

Faculty Remarks: (Keep this blank for faculty)

Practical - 2

Practical Title: Semantic HTML5 Pages with Accessibility-Ready Structure

Problem Definition: Develop static HTML5 skeletons for at least 10 StudentHub pages such as Home, About, Register, Login, Dashboard, Events, Profile, Contact, Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.

A. Theory

1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?

   Answer: Yes, Here it's a short descriptive part on it

- **<header>**: Contains introductory content, logos, navigation, or search bars for a page or section.
- **<nav>**: Wraps the primary navigation links of the website.
- **<main>**: Holds the unique, dominant content of the page. Only use one per page.
- **<section>**: Groups related content together under a single theme, usually requiring a heading (<h2>-<h6>).
- **<article>**: Encloses self-contained content that makes sense on its own (e.g., blog posts, product cards).
- **<aside>**: Holds indirectly related content, like sidebars, callout boxes, or advertising.
- **<footer>**: Contains closing information, such as copyright data, contact info, or secondary links.

https://dev.to/sharique_siddiqui_8242dad/semantic-html-header-footer-nav-section-article-aside-main-2480

2. Are form labels, headings, image alt text, and navigation links accessible?

Answer:

Yes, form labels, headings, image alt text, and navigation links are highly accessible when built using semantic HTML. They serve as the core foundational pillars of web accessibility, allowing users with visual, cognitive, or physical impairments to successfully navigate digital spaces using assistive technologies like screen readers.

3. Is the page structure consistent across all pages?

Answer:

Yes, my page structure is completely consistent across all pages**.** Every page uses the exact same header, main content layout, navigation structure, and footer to ensure a predictable and fully accessible user experience .

B. Result **(Keep only one option which is suitable to practical)**

The required StudentHub pages were successfully created using semantic HTML5 elements and an accessibility-ready structure. The pages provide a consistent navigation and content structure across the portal.

C. Evidence of Work (Screenshots)

D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

During the implementation of the StudentHub pages, the following challenges were faced:

- Understanding the correct use of semantic HTML5 tags instead of using elements for everything.
- Maintaining a consistent structure and navigation across multiple pages.
- Creating proper form labels and accessible input fields.
- Maintaining a logical heading hierarchy on different pages.

- Adding meaningful alternative text to images.
- Managing common navigation and footer elements across all pages.
- Ensuring that the HTML structure remains clean and easy to maintain.

These challenges were solved by referring to HTML5 semantic elements, following accessibility practices, and maintaining a common page structure across the StudentHub portal.

F. Learning Outcomes:

After completing this practical, I learned:

- The purpose and proper usage of semantic HTML5 elements.
- How to structure webpages using <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>.
- How to create accessibility-friendly forms using labels and appropriate input elements.
- The importance of a logical heading hierarchy.
- How to use descriptive alt text for images.
- How to create consistent navigation across multiple webpages.
- How semantic HTML improves website structure, readability, and accessibility.
- How to organize multiple static HTML pages for a larger web project.

Faculty Remarks:

Practical - 3

Practical Title: Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

Problem Definition: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

A. Theory

1. How does the layout adapt for mobile, tablet, and desktop screens?
   Answer:
   Responsive web layouts adapt across devices using fluid grids and CSS media queries to automatically rearrange content based on available screen space. On large desktop screens, content typically spans multiple columns with fully expanded navigation menus. As the screen shrinks to a tablet size, the layout usually condenses into fewer columns with larger, touch-friendly buttons. Finally, on small mobile screens, the design collapses into a single vertical column for easy scrolling, and wide navigation bars are tucked away behind a compact "hamburger" menu icon to maximize horizontal real estate.

   https://www.w3schools.com/css/css_rwd_intro.asp

2. Which layout technique is used and why?
   Answer:
   In responsive UI design, these techniques are typically used together as a combined strategy. CSS Grid handles the overall two-dimensional page structure (like dashboard layouts and galleries) because it manages both rows and columns simultaneously. Flexbox controls the one-dimensional alignment inside individual components (like navigation bars or button rows) because it excels at distributing space and centering content along a single axis. Frameworks like Bootstrap or Tailwind are used to apply these Grid and Flexbox layouts rapidly via pre-built utility classes, ensuring quick development and consistent responsive breakpoints without writing custom CSS media queries manually.

https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout/Relationship_with_other_layout_methods

3. Are typography, spacing, colors, and contrast readable?
Answer:
Yes. Typography, spacing, colors, and contrast are selected to make the StudentHub interface easy to read and navigate.

Readable font sizes are used for headings and normal text. Proper spacing is maintained between sections, cards, buttons, and form elements. The color combination provides sufficient contrast between the background and text. Important buttons and interactive elements are also visually distinguishable.

The same design system is maintained across the different pages to provide a consistent user experience.

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics

4. Is the framework used consistently without unnecessary inline CSS?
Answer:
Yes. If Bootstrap is used in the StudentHub project, its components and utility classes are used consistently wherever they are useful.

Custom CSS is used for project-specific styling that is not provided by the framework. Unnecessary inline CSS is avoided because keeping styles in CSS files makes the project easier to maintain and update.

The framework and custom CSS are organized so that the design remains consistent across all pages.

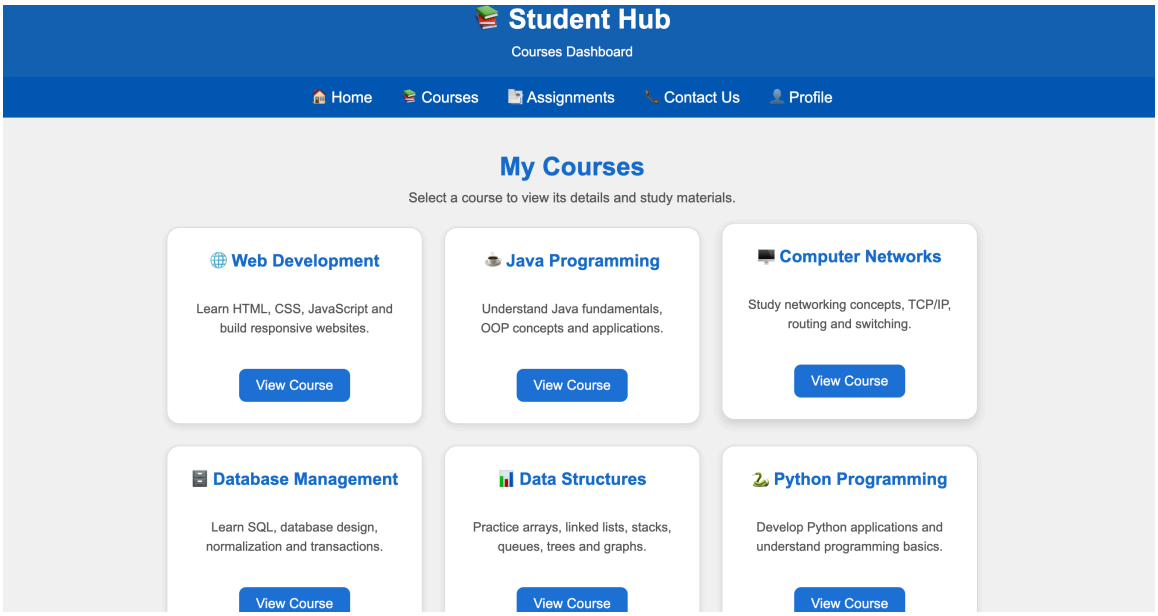
https://getbootstrap.com/docs/

B. Result

The StudentHub pages were successfully designed with responsive layouts. CSS Grid was used for structured content layouts, while Flexbox was used for navigation and onedimensional arrangements.

The interface adapts to mobile, tablet, and desktop screen sizes while maintaining consistent typography, spacing, colors, and overall design.

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

During the implementation of the responsive StudentHub interface, the following challenges were faced:

- Understanding how CSS Grid and Flexbox should be used for different layouts.
- Making the same page work correctly on different screen sizes.
- Adjusting the number of columns for cards and dashboard sections.

- Preventing text, buttons, and navigation elements from overflowing on mobile screens. 5. Maintaining consistent spacing and typography across different devices.
- Using media queries correctly for tablet and mobile breakpoints.
- Keeping custom CSS organized without depending too much on inline styling.

These challenges were solved by testing the pages using the browser's responsive design mode and adjusting the Grid, Flexbox, spacing, and media-query rules accordingly**.**

F. Learning Outcomes:

After completing this practical, I learned:

- How to create responsive web layouts using CSS3.
- How to use Flexbox for arranging elements in one direction.
- How to use CSS Grid for row and column-based layouts.
- How media queries help websites adapt to different screen sizes.
- How to follow a mobile-first approach when designing webpages.
- How to maintain consistent typography, spacing, colors, and contrast.
- How to test responsive webpages using browser developer tools.
- How to organize CSS so that it can be reused across multiple pages.
- How responsive design improves the usability of websites on different devices.

Faculty Remarks:

Practical - 4

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1. How are DOM elements selected and modified?

Answer:

DOM elements are selected using JavaScript document methods like querySelector() and modified by updating their properties, styles, or structural methods. This process forms the foundation of dynamic web development

https://www.geeksforgeeks.org/javascript/how-to-select-dom-elements-in-javascript/

2. Are event listeners attached correctly?

Answer:

Yes. Event listeners are attached using addEventListener(). In our website, event listeners are used for actions such as the theme toggle and registration form submission. This allows the webpage to respond to user actions without putting JavaScript directly inside HTML elements.

Reference: W3C – Web Programming Guidelines

3.Is local Storage used for remembering theme choice?

Answer:

Yes ,local storage is widely used by websites to remember user choices and preferences

https://cookie-script.com/blog/local-storage-and-session-storage

4.Does interactivity improve usability without breaking accessibility?

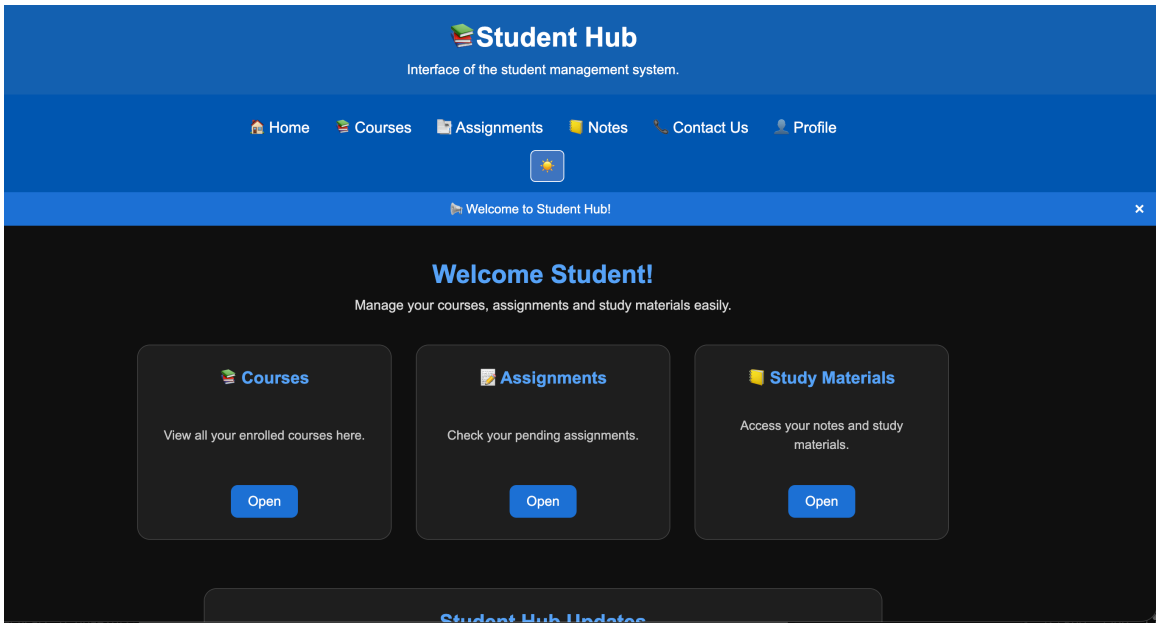
Answer:

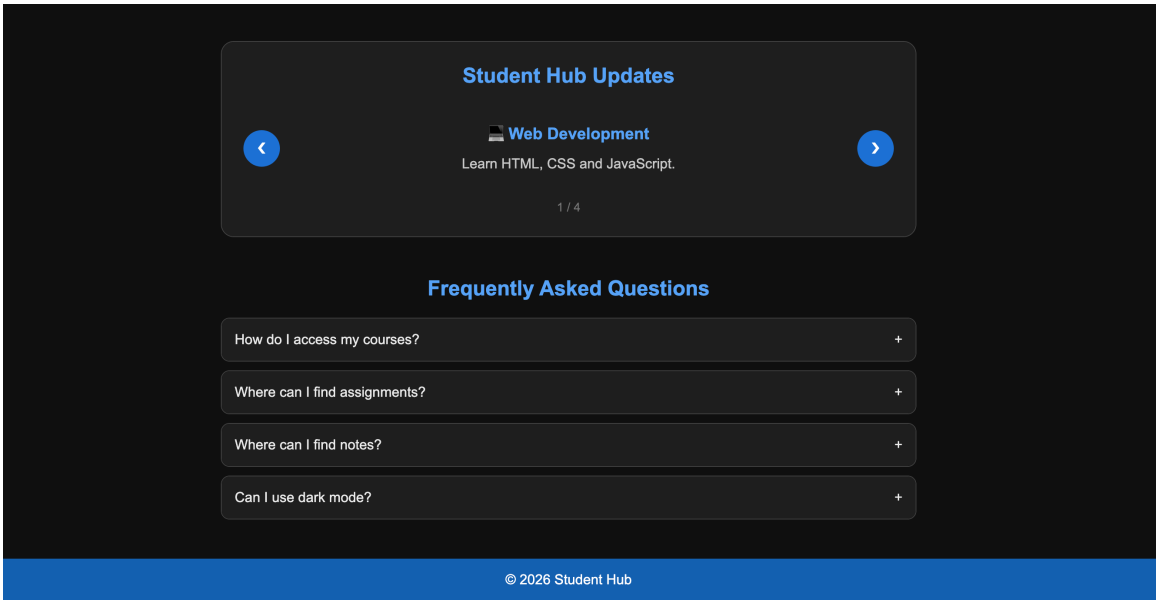
Yes, well-designed interactivity improves overall usability without breaking accessibility, provided interactive components are built to support assistive technologies and standard navigation rules.

B. Result

The StudentHub pages were successfully changing its theme from dark mode to light mode perfectly without any concerns

C. Evidence of Work (Screenshots)





D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

When I made the dark and light mode the code and all were perfect but still button was not working at all as if it was unclickable then I added pop up notification with button as button is clicked then it started working .

F. Learning Outcomes:

After this practical I learn how to use java script toggle the screen add drop down menu and etc.

Faculty Remarks:

Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

1. **Are correct input types and attributes used?**
   Answer:
   Yes. The form uses appropriate input types such as email, tel, and password. Attributes such as name, id, and value are also used so that the form data can be identified and processed correctly. HTML input types also help browsers provide suitable validation and input controls.

   Reference: W3C – Validating Input

2. **Are validation errors displayed near the relevant fields?**
   Answer:
   Yes. Validation errors are displayed near their respective fields using elements such as nameError, emailError, mobileError, and passwordError. This makes it easier for users to identify and correct the specific input that contains an error.

   Reference: W3C – Validating Input

3. **Is password strength checked?**
   Answer:
   Yes. The password is checked using a regular expression. It must contain at least 8 characters, including an uppercase letter, lowercase letter, number, and special character. This ensures that weak passwords are rejected during form validation.

   Reference: W3C – Form Instructions

4. Is the form accessible using keyboard and screen-reader-friendly labels?

   Answer:

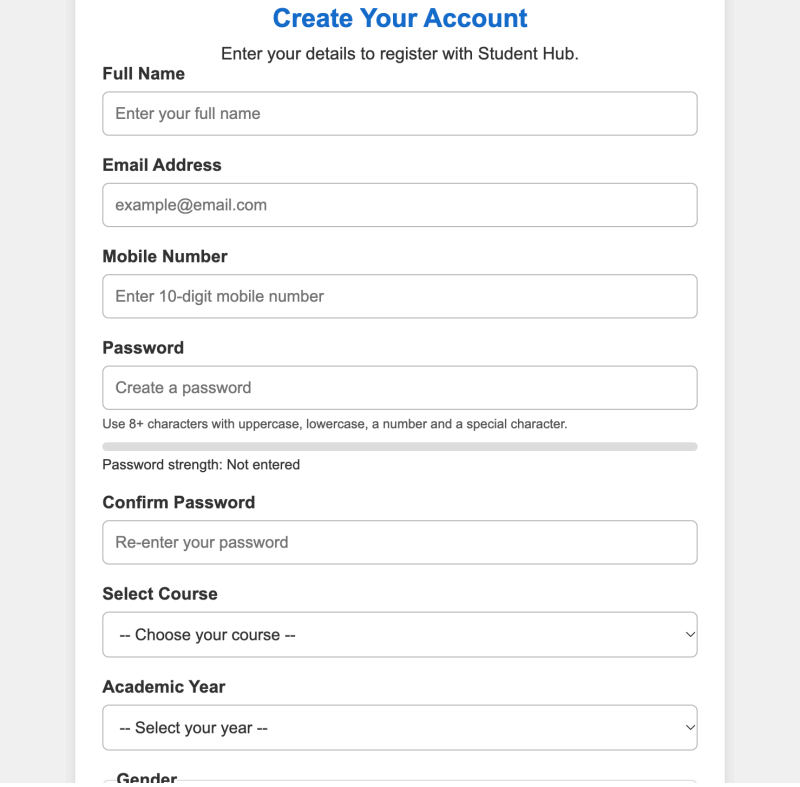
   The form uses standard HTML controls such as inputs, radio buttons, checkboxes, select menus, and a submit button, which can be operated using the keyboard. It also provides visible labels for the form fields. For better screen-reader accessibility, each <label> should be explicitly associated with its input using matching for and id attributes.

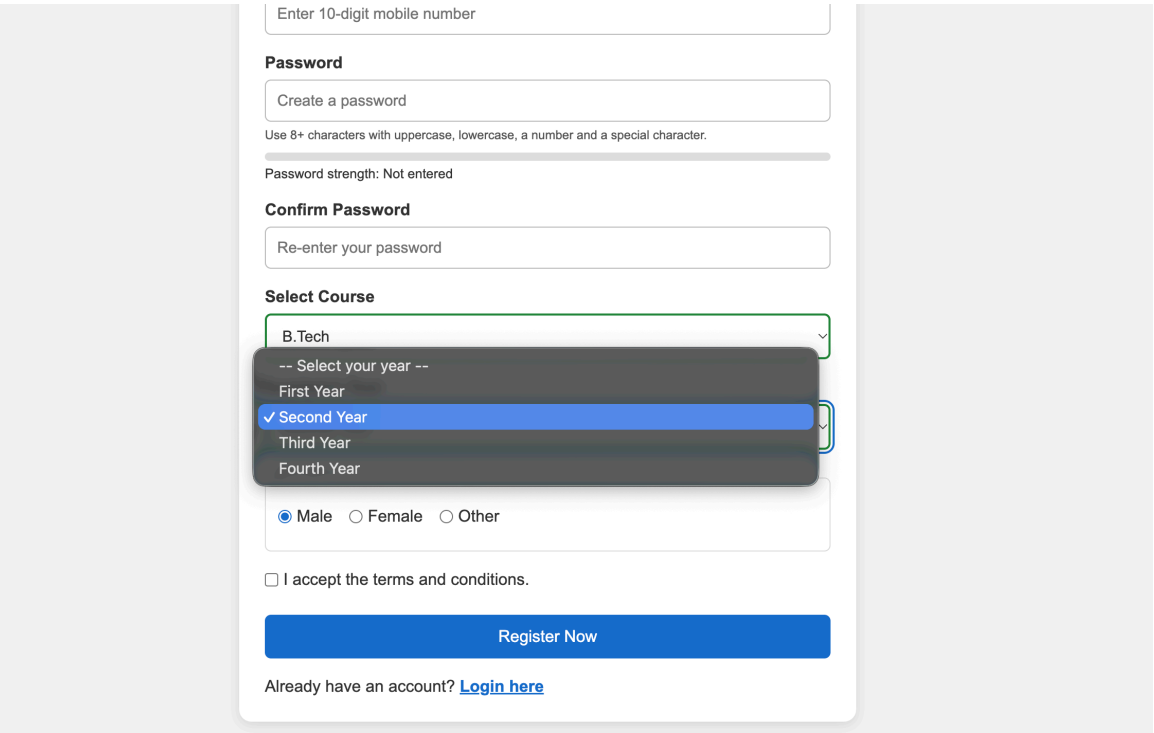
   Reference: W3C – Labeling Controls

B. Result

The StudentHub registration page is now more accurate using javascript and drop down and other tags using regex in email and passwords to store proper data

C. Evidence of Work (Screenshots)





D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

In this practical the challenges I faced were the regex when I was entering the regex expression it still wasn't detecting it and the issue was the id name in html and javascript had a spelling difference

F. Learning Outcomes:

After this practical I learn how to use diff javascript concepts such data entering in file so that it can radio buttons dropdown menu and regex expressions how to create them

Faculty Remarks:

Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. **How is JSON fetched, parsed, and rendered?**

   The JSON data is fetched using the fetch() method from the events.json file. The response.json() method parses the JSON response into JavaScript data. The data is then stored in the events array and rendered dynamically using forEach() and innerHTML.

2. **Which array methods are used for search, filter, sort, and pagination?**

   The filter() method is used for searching events by title and filtering events by category. The sort() method is used for sorting events by name or date. The slice() method is used for pagination to display a specific number of events on each page.

3. **How are loading and error states handled?**

   While the JSON data is being loaded, a loading message is displayed. If the fetch request fails or the response is not successful, an error is generated using throw new Error(), and the catch() block displays an error message to the user.

4. **How is modularity maintained in JavaScript files?**

   Modularity is maintained by keeping the event-related functionality in a separate data.js file. Functions such as displayEvents(), createPagination(), and changePage() handle specific tasks, making the code easier to understand, maintain, and modify.

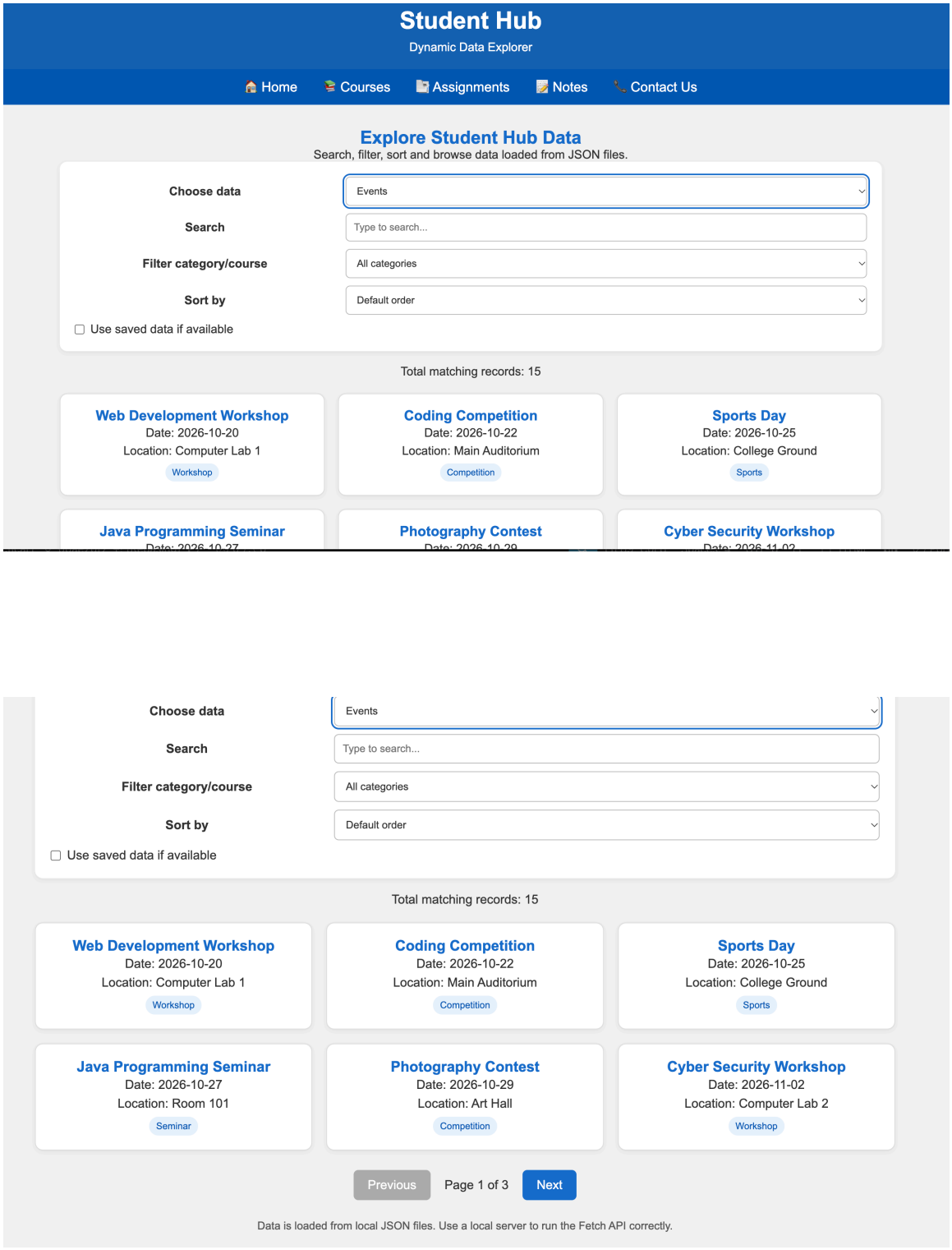
Reference: MDN – Fetch API

Reference: MDN – Array methods

B. Result

In this practical I used json (java script object notation) and fetched data using javascript in file and using it to search the objects

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

While doing this practical, I faced some small issues with fetching the JSON file and connecting the JavaScript file correctly. I also had to understand how filtering, sorting, and pagination work together. Fixing these issues helped me understand the Fetch API and JavaScript array methods better.

F. Learning Outcomes:

This practical helped us understand how to fetch and display JSON data using JavaScript. We also learned to add search, filter, sorting, and pagination features and handle loading and error messages.

Faculty Remarks:

Practical 7

PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements.

A. Theory

1. Is the form submitted using POST?

Answer:

Yes, the form uses the POST method to send user data to the PHP server without displaying the submitted values in the URL.

https://www.w3schools.com/php/php_forms.asp

2. Are inputs validated and sanitized server side?

Answer:

Yes, the PHP script validates the submitted data and sanitizes appropriate inputs to prevent invalid data from being processed or stored.

https://www.w3schools.com/php/php_form_validation.asp

3.Is file writing handled safely?

Answer:

Yes, file writing is handled using PHP file-handling functions, with appropriate checks for file access and writing errors. CSV or JSON formats are used to store the submitted records.

https://www.w3schools.com/php/php_file_handling.asp

4.Are success and error responses displayed clearly?

Answer:

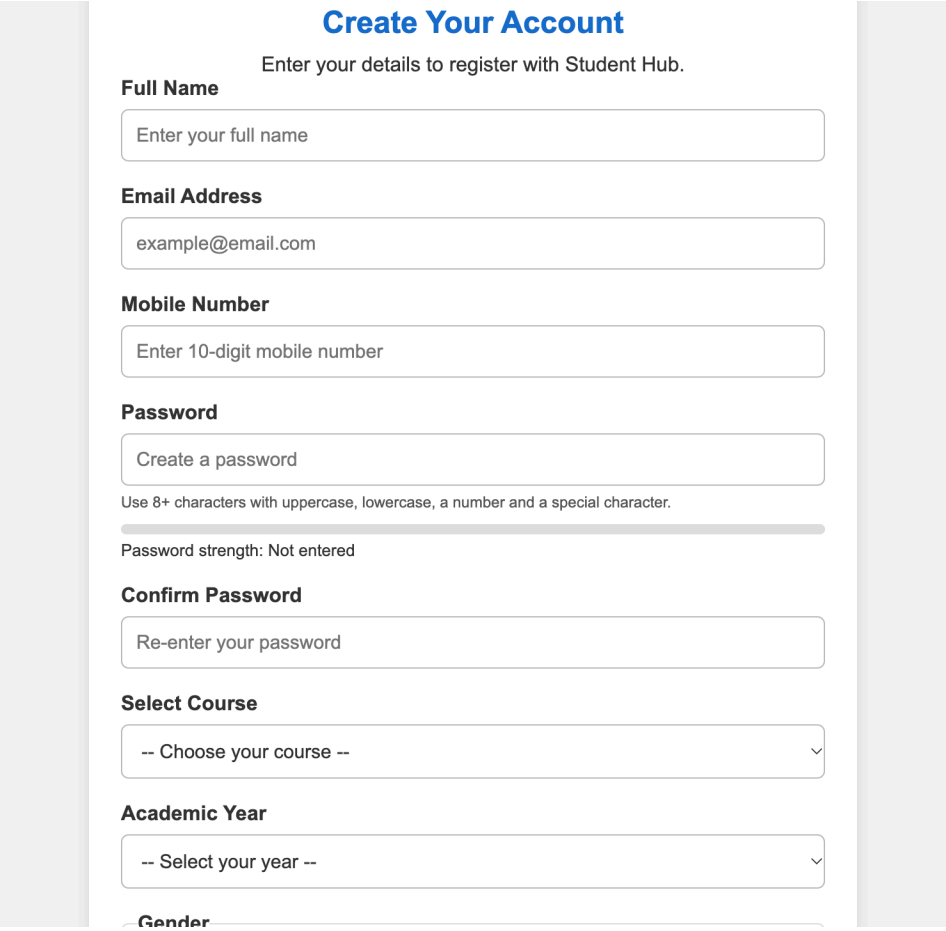
Yes, appropriate success messages are displayed when the data is stored successfully, while error messages inform users about invalid inputs or file-writing failures.

https://www.w3schools.com/php/php_form_validation.asp

B. Result

Thus, PHP form processing was implemented using the POST method, server-side validation and sanitization, and CSV/JSON file storage. Appropriate success and error messages were provided to ensure reliable and user-friendly form processing.

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

The main challenges faced were handling form data using POST, validating and sanitizing user inputs, managing file permissions, storing records correctly in CSV/JSON format, and displaying suitable success and error messages.

F. Learning Outcomes:

This practical helped in understanding PHP form processing, handling POST requests, performing server-side validation and sanitization, writing data safely to CSV/JSON files, managing file-related errors, and displaying clear responses to users. It also introduced the importance of secure data handling in backend applications.

Faculty Remarks:

Practical 8

MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements.

A. Theory

1. Is the schema normalized properly?

Answer:

Yes, the StudentHub database is normalized by separating the data into three tables: students, events, and registrations. Each table stores specific information, reducing data duplication and maintaining data consistency. The registrations table connects students with events and helps implement a many-to-many relationship.

https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/

2. Are primary keys and foreign keys correctly defined?

Answer:

Yes, primary keys uniquely identify each record in their respective tables. The student_id in the students table and event_id in the events table are primary keys. The registrations table contains student_id and event_id as foreign keys referencing their respective tables. These constraints maintain referential integrity.

https://www.geeksforgeeks.org/dbms/difference-between-primary-key-and-foreign-key/

3.Is the database connection handled securely?

Answer:

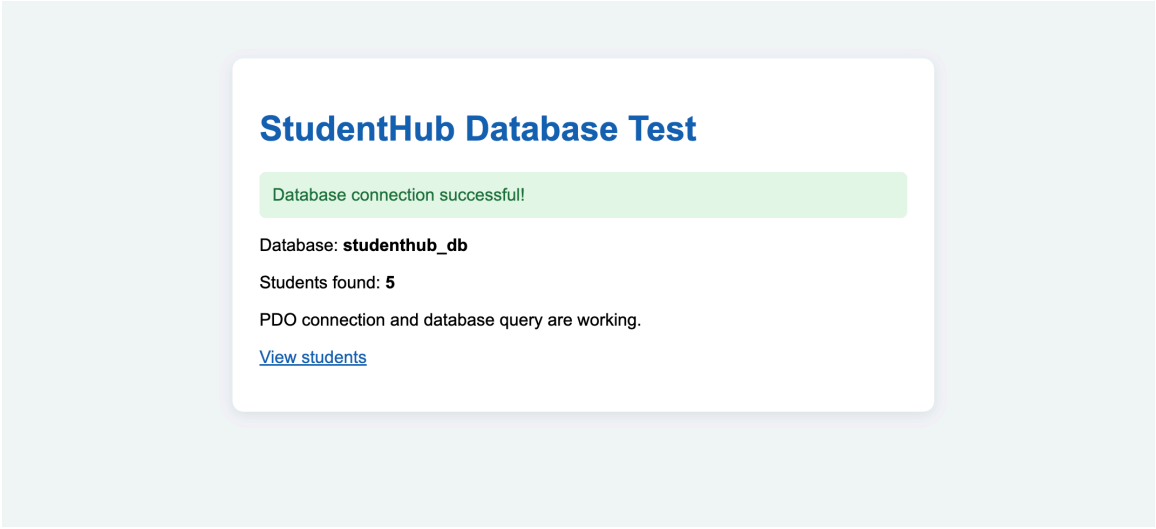
Yes, the database connection is handled securely using PHP PDO. Database credentials are kept in a separate db.php file, exceptions are used for error handling, and prepared statements are used to execute SQL queries safely. Prepared statements help prevent SQL injection attacks.

https://www.geeksforgeeks.org/php/php-data-objects-pdo/

B. Result

Thus, the StudentHub database was designed using MySQL with properly normalized tables, primary and foreign key constraints, and secure PHP PDO connectivity. Prepared statements help prevent SQL injection and ensure safe database operations.

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

Designing a normalized database structure for students, events, and registrations. Establishing relationships between tables using primary keys and foreign keys.Connecting PHP to MySQL using PDO.

F. Learning Outcomes:

Understanding how to design a normalized MySQL database for the StudentHub application, creating tables with primary and foreign keys, managing databases using phpMyAdmin, establishing a secure database connection using PHP PDO, and implementing prepared statements to prevent SQL injection and maintain data integrity.

Faculty Remarks:

Practical 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/ username, hash passwords using password_hash(), and display
appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?

Answer:

Yes, the password_hash() function is used to convert the user's password into a secure hash before storing it in the database. This helps protect passwords from being exposed if the database is compromised.

https://www.geeksforgeeks.org/php/php-password_hash-function/

2. Is duplicate email/username checked?

Answer:

Yes, the system checks whether the email address or username already exists in the database before inserting a new user. This prevents duplicate accounts and helps maintain data consistency.

https://www.geeksforgeeks.org/php/how-to-use-prepared-statements-in-php/?

3. Are frontend and backend validations consistent?

Answer:

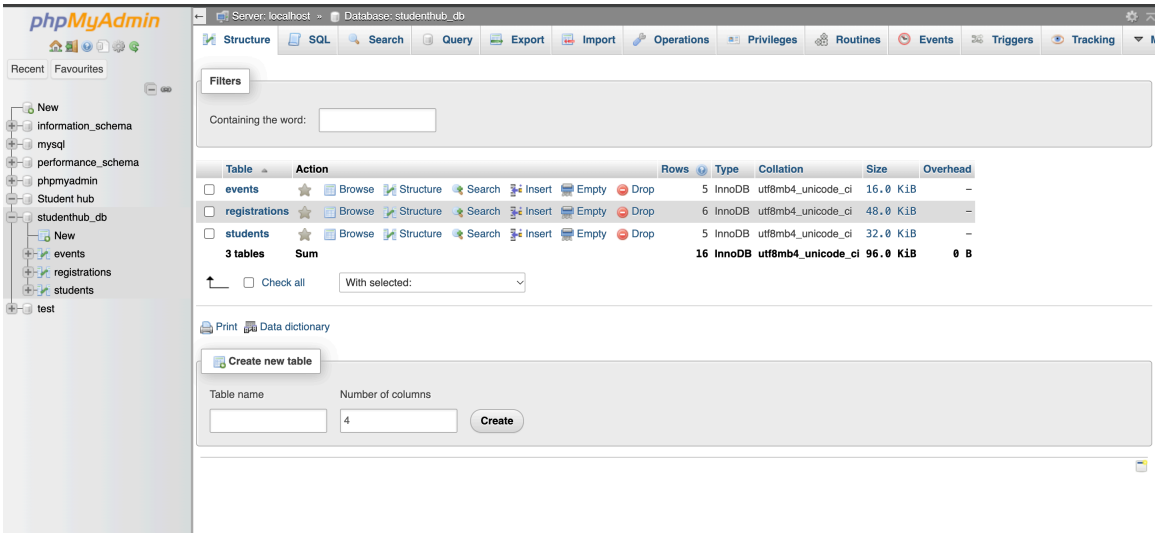
Yes, frontend validation provides immediate feedback to users, while backend validation checks the submitted data before saving it to the database. Both validations should enforce the same rules for required fields, valid email addresses, and password requirements.

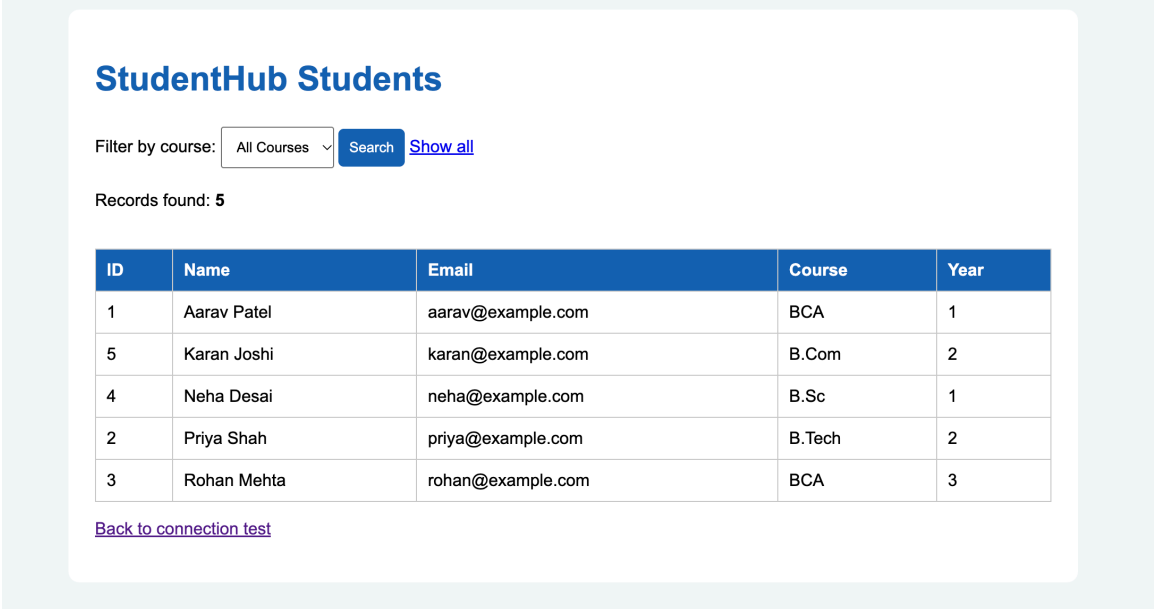
https://www.w3schools.com/php/php_form_validation.asp/

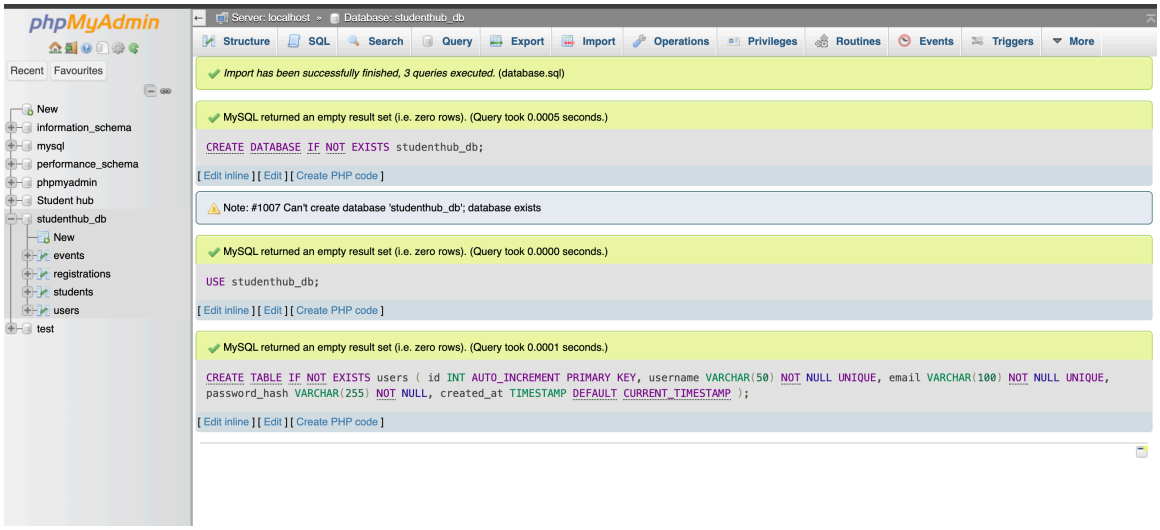
B. Result

Thus, secure user registration was implemented using PHP and MySQLi with input validation, duplicate email and username checks, password hashing, and database insertion to protect user information and maintain data integrity.

C. Evidence of Work (Screenshots)







D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

The main challenges faced were implementing consistent frontend and backend validation, checking for duplicate email addresses and usernames, understanding password hashing, connecting PHP to MySQL, and handling database errors during registration.

F. Learning Outcomes:

The practical helped in understanding secure user registration using PHP and MySQLi, validating user inputs on both frontend and backend, checking duplicate email addresses and usernames, hashing passwords using `password_hash()`, inserting user details into the database, and applying basic web security practices to protect user information.

Faculty Remarks:

Practical 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions.Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session
timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Answer:

Yes, PHP sessions are started using session_start(). After successful login, the session ID is regenerated using session_regenerate_id(true). During logout, session data is cleared, the session cookie is invalidated, and the session is destroyed.

https://www.w3schools.com/php/php_sessions.asp

2. Is password_verify() used correctly?

Answer:

Yes, password_verify() checks the entered password against the password hash stored in the database. This allows authentication without storing plain-text passwords.

https://www.w3schools.com/php/func_crypt_password_verify.asp

3.Is unauthorized dashboard access blocked?

Answer:

Yes, dashboard pages check whether the user has a valid login session before displaying protected content. If a user is not authenticated, they are redirected to the login page, preventing unauthorized access

https://www.geeksforgeeks.org/php/how-to-redirect-a-user-to-the-registration-if-he-has-not-logged-in/

4.Is session timeout implemented?

Answer:

Yes, a session timeout is implemented by tracking the user's last activity time. If the user remains inactive beyond the configured time limit, the session is invalidated and the user must log in again.

https://www.geeksforgeeks.org/php/php-sessions/

4.Are admin and student users redirected to appropriate dashboards?

Answer:

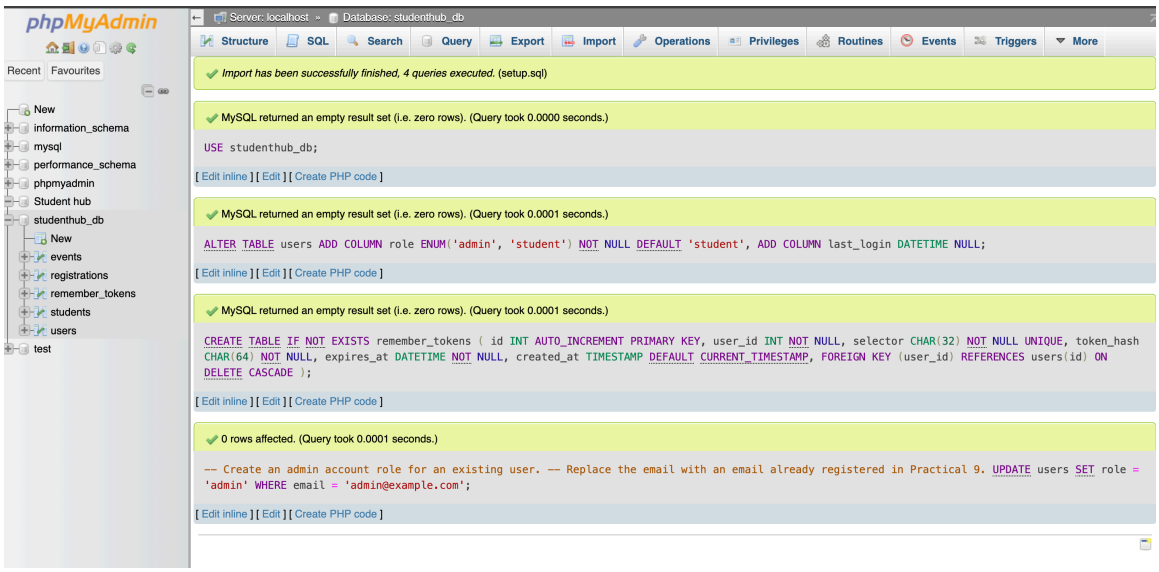
Yes, role-based access control checks the user's assigned role after successful authentication. Administrators are redirected to the admin dashboard, while students are redirected to the student dashboard. Each dashboard must also verify the user's role to prevent unauthorized access

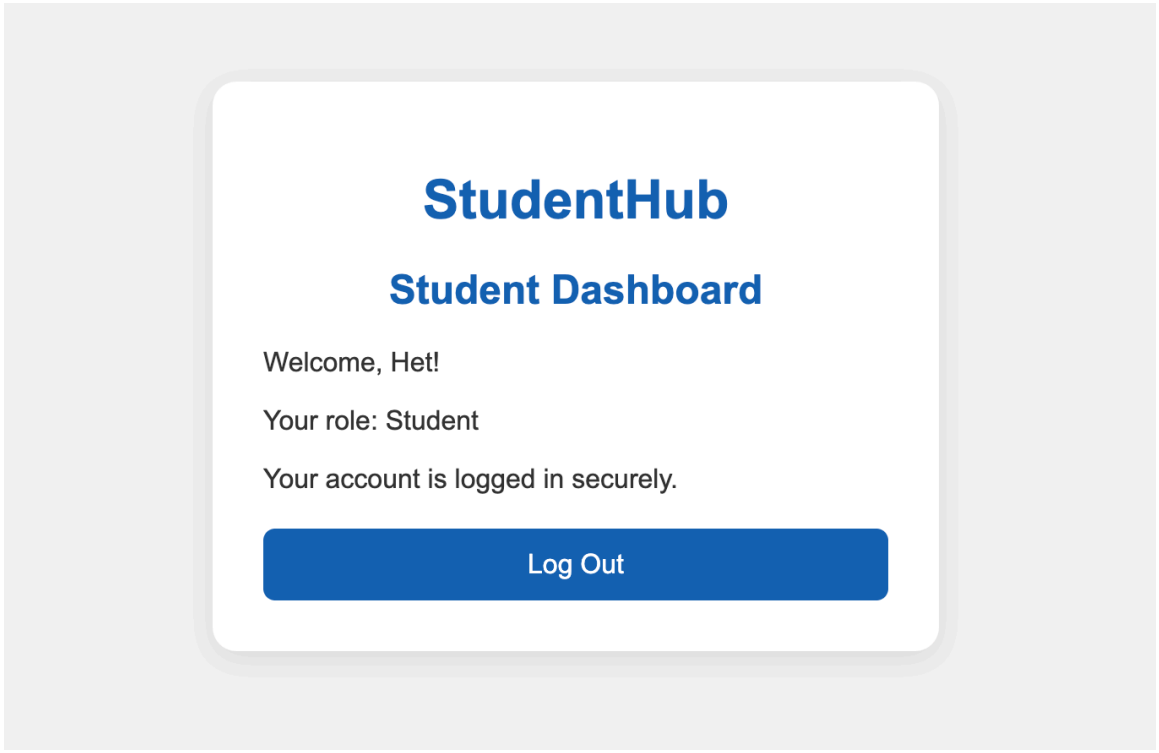
https://www.geeksforgeeks.org/ethical-hacking/role-based-access-control-1/

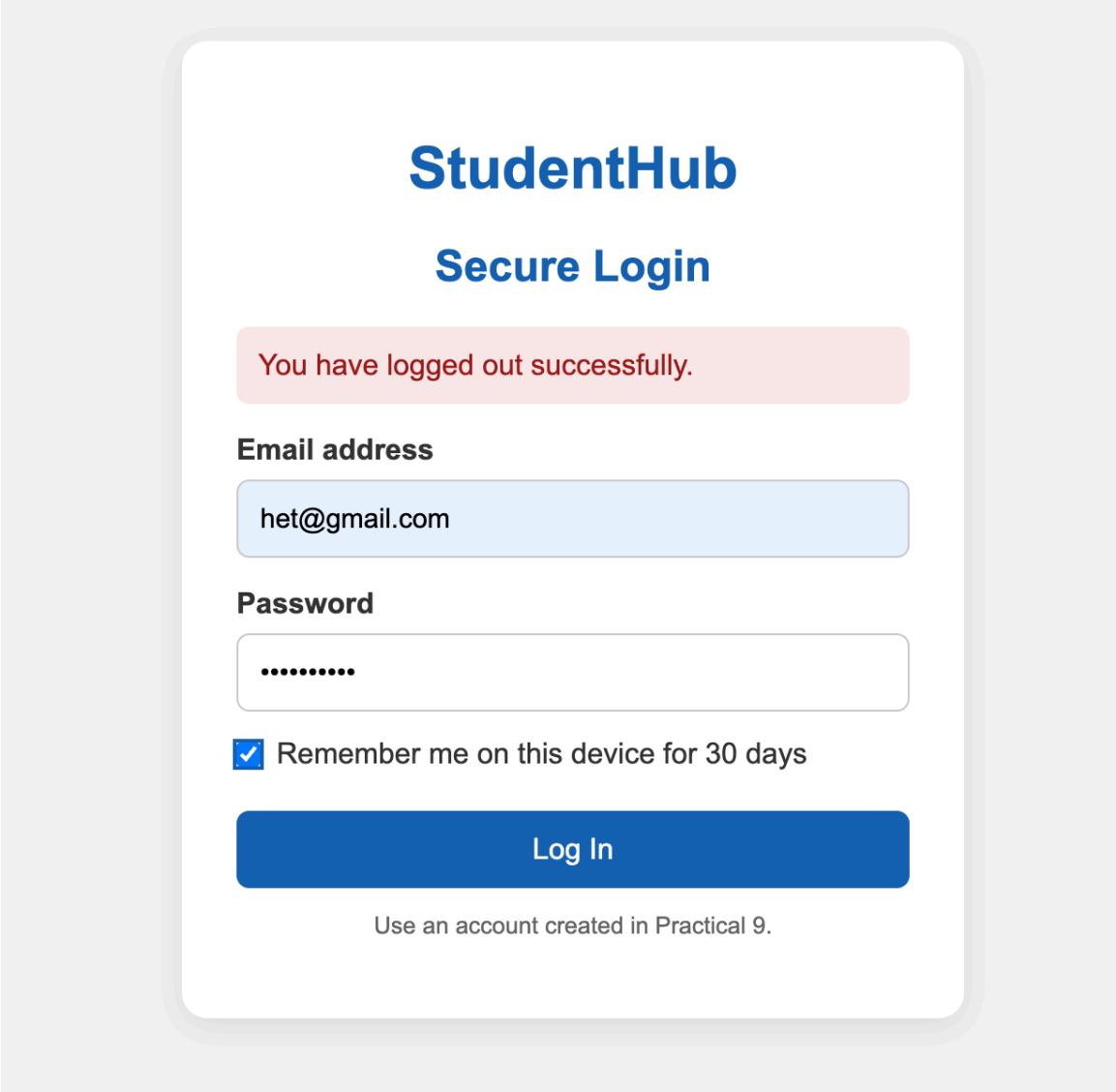
B. Result

Thus, secure login and logout functionality was implemented using PHP sessions and password verification. Session ID regeneration, dashboard access restrictions, session timeout, and role-based redirection help improve authentication security and prevent unauthorized access.

C. Evidence of Work (Screenshots)







D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

The main challenges faced were managing PHP sessions securely, verifying passwords using password_verify(), preventing unauthorized dashboard access, implementing session timeout, and redirecting admin and student users to their respective dashboards.

F. Learning Outcomes:

This practical helped in understanding secure authentication using PHP sessions, password verification, session ID regeneration, logout handling, access restrictions, session timeout, and role-based access control. It also introduced the importance of secure cookie management and remember-me tokens in developing reliable web authentication systems.

Faculty Remarks:

Practical-11

Student Management CRUD with Search and Filter

Theory :

1. **Are SELECT, INSERT, UPDATE, and DELETE operations working?**

Answer: Yes, all four database operations are working correctly to view, add, update, and delete records.

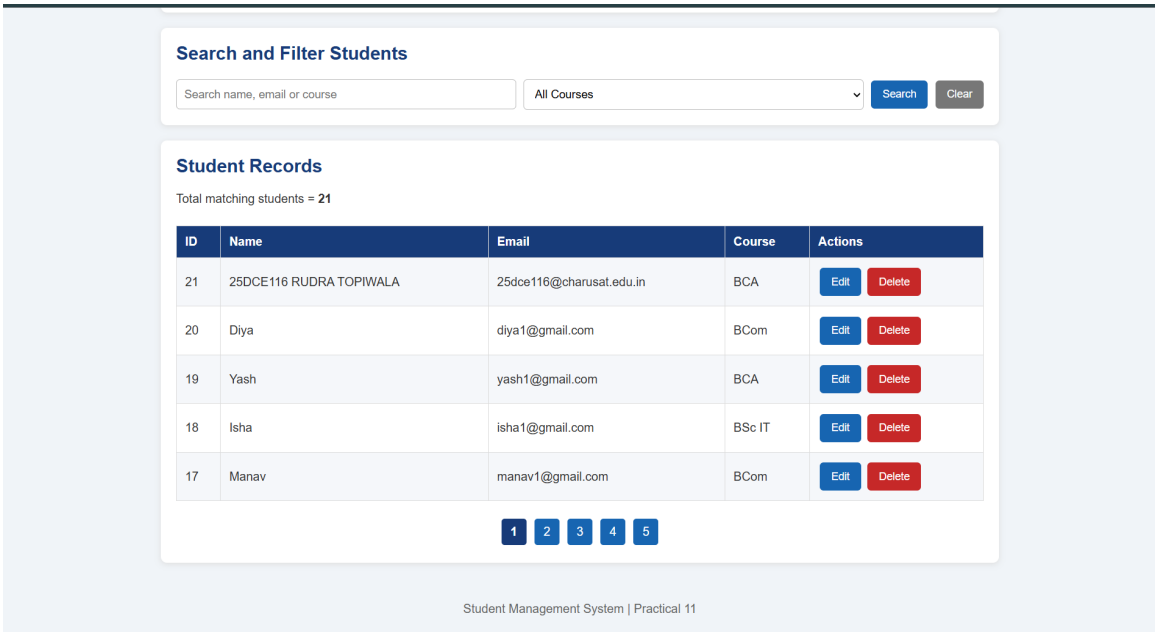
2. **Is search by name/email/course implemented?**

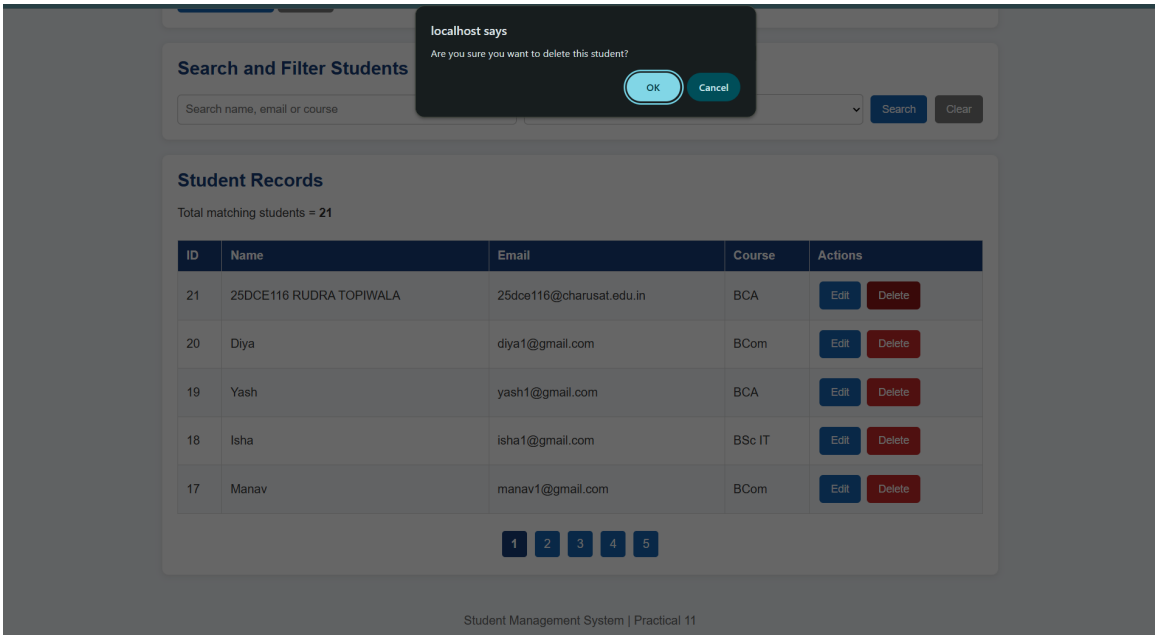
Answer: Yes, the search functionality allows users to find records using name, email, or course.

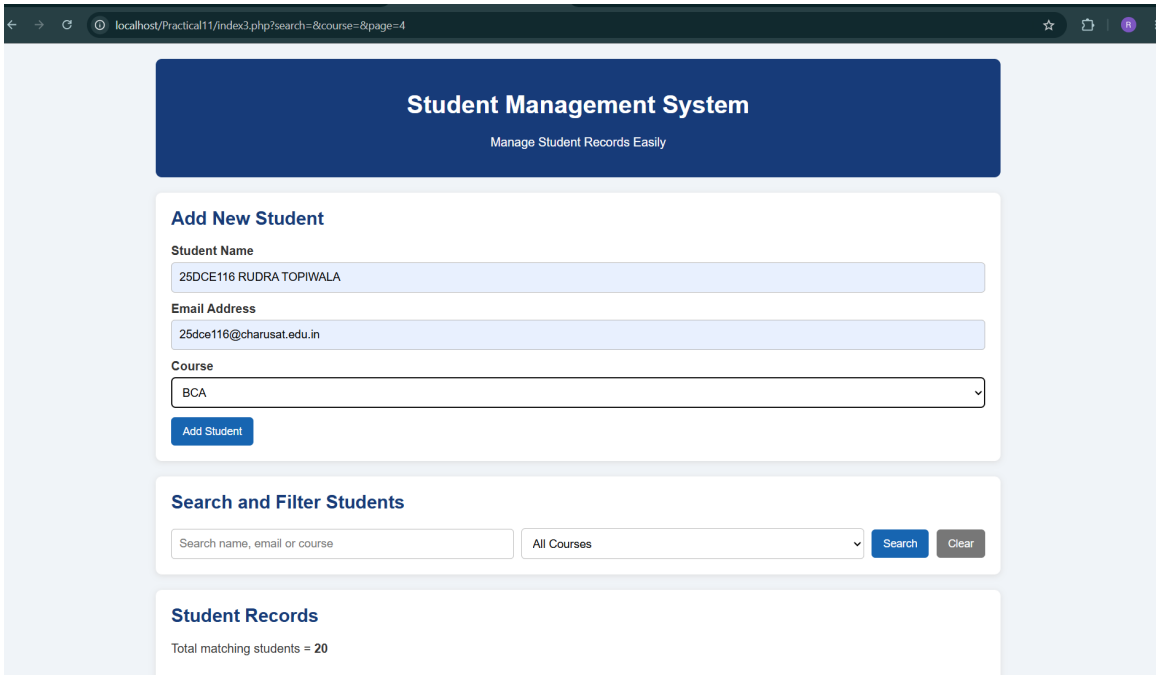
3. **Are delete operations confirmed before execution?**

Answer: Yes, a confirmation message is displayed before deleting a record to prevent accidental deletion.

Result:


Search and Filter Students
Search name, email or course
All Courses
Search
Clear
Student Records
Total matching students = 21
ID
Name
Email
Course
Actions
21
25DCE116 RUDRA TOPIWALA
25dce116@charusat.edu.in
BCA
Edit
Delete
20
Diya
diya1@gmail.com
BCom
Edit
Delete
19
Yash
yash1@gmail.com
BCA
Edit
Delete
18
Isha
isha1@gmail.com
BSc IT
Edit
Delete
17
Manav
manav1@gmail.com
BCom
Edit
Delete
1
2
3
4
5
Student Management System | Practical 11


localhost says
Are you sure you want to delete this student?
OK
Cancel
Search and Filter Students
Search name, email or course
Search
Clear
Student Records
Total matching students = 21
ID
Name
Email
Course
Actions
21
25DCE116 RUDRA TOPIWALA
25dce116@charusat.edu.in
BCA
Edit
Delete
20
Diya
diya1@gmail.com
BCom
Edit
Delete
19
Yash
yash1@gmail.com
BCA
Edit
Delete
18
Isha
isha1@gmail.com
BSc IT
Edit
Delete
17
Manav
manav1@gmail.com
BCom
Edit
Delete
1
2
3
4
5
Student Management System | Practical 11



Git Hub Repository Link:

https://github.com/Dhanraj2408/StudentHub

Challenges Faced: I faced difficulties in implementing CRUD operations using PHP and MySQL. I struggled with implementing search and filter functionality for student records. I also faced challenges in handling delete confirmations and displaying success or failure messages.

Learning Outcome:- I learned to perform INSERT, SELECT, UPDATE, and DELETE operations using PHP and MySQL.
I understood how to implement search and filter functionality for student records. I also learned to handle user confirmations and display appropriate messages after database operations.

Practical-12

Event Management CRUD with Poster Upload and File Validation

Theory :

1. **Are add, update, and delete functionalities implemented correctly?**

Answer: Yes, users can add, update, and delete event records correctly.

2. **Is the UI linked with the database correctly?**

Answer: Yes, the user interface is connected to the database using PHP and MySQL.

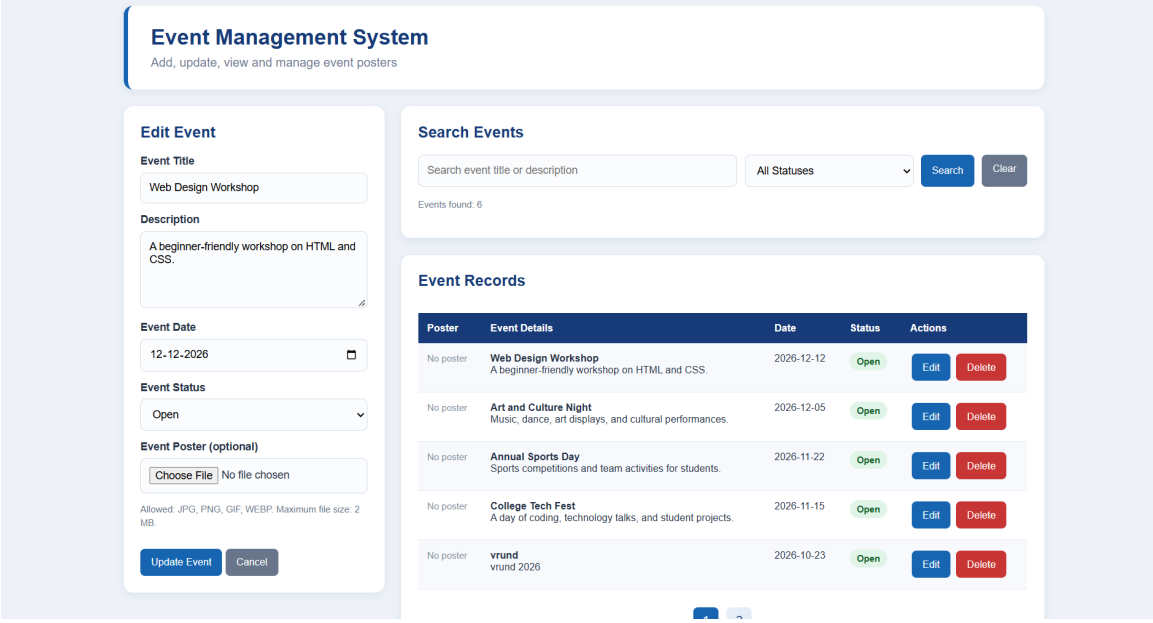
3. **Are poster uploads validated for type and size?**

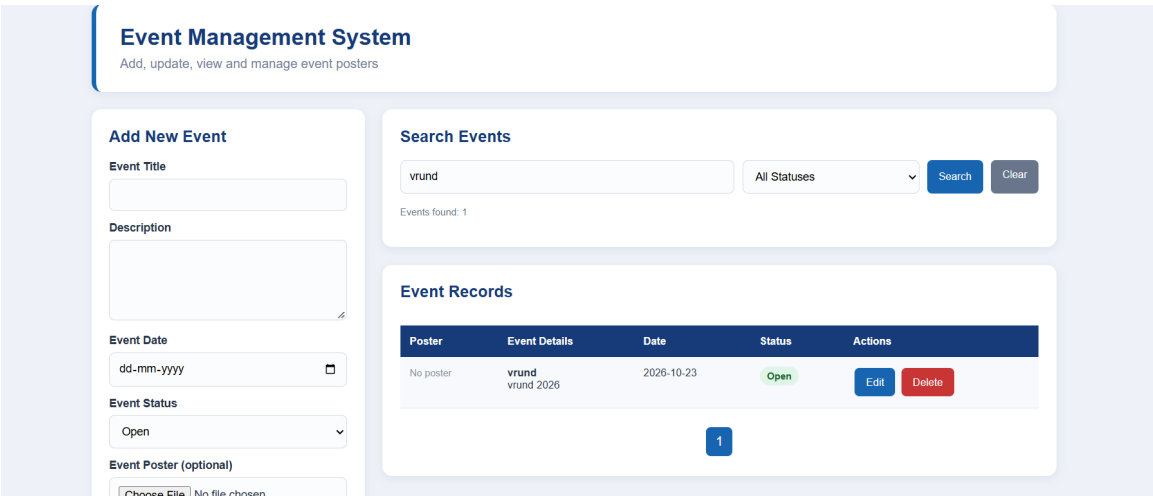
Answer: Yes, poster uploads are validated to allow supported file types and sizes.

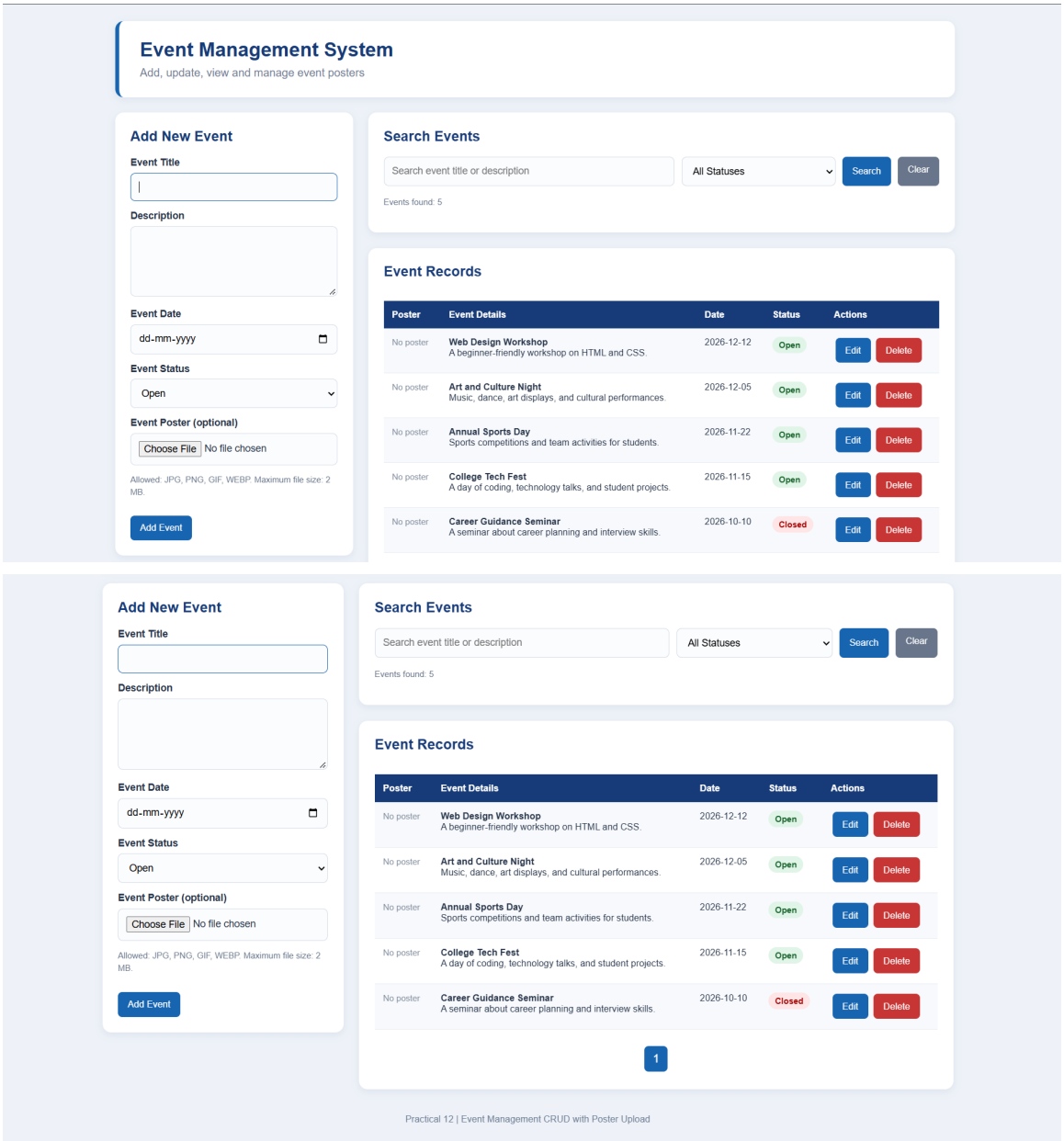
4. **Are success and failure messages displayed?**

Answer: Yes, appropriate success and failure messages are displayed after each operation.

Result:


Event Management System
Add, update, view and manage event posters
Edit Event
Event Title
Web Design Workshop
Description
A beginner-friendly workshop on HTML and CSS.
Event Date
12-12-2026
Event Status
Open
Event Poster (optional)
Choose File No file chosen
Allowed: JPG, PNG, GIF, WEBP. Maximum file size: 2 MB.
Update Event
Cancel
Search Events
Search event title or description
All Statuses
Search
Clear
Events found: 6
Event Records
Poster
Event Details
Date
Status
Actions
No poster
Web Design Workshop
A beginner-friendly workshop on HTML and CSS.
2026-12-12
Open
Edit
Delete
No poster
Art and Culture Night
Music, dance, art displays, and cultural performances.
2026-12-05
Open
Edit
Delete
No poster
Annual Sports Day
Sports competitions and team activities for students.
2026-11-22
Open
Edit
Delete
No poster
College Tech Fest
A day of coding, technology talks, and student projects.
2026-11-15
Open
Edit
Delete
No poster
vrund
vrund 2026
2026-10-23
Open
Edit
Delete


Event Management System
Add, update, view and manage event posters
Add New Event
Event Title
Description
Event Date
dd-mm-yyyy
Event Status
Open
Event Poster (optional)
Search Events
vrund
All Statuses
Search
Clear
Events found: 1
Event Records
Poster
Event Details
Date
Status
Actions
No poster
vrund
vrund 2026
2026-10-23
Open
Edit
Delete
1



Git Hub Repository Link:

https://github.com/Dhanraj2408/StudentHub

Challenges Faced: I faced difficulties in implementing event CRUD operations using PHP and MySQL. I struggled with uploading event posters and validating their file type and size. I also faced challenges in connecting the UI with the database and displaying appropriate messages.

Learning Outcome:- I learned to add, view, update, and delete event records using PHP and MySQL. I understood how to upload posters with proper file type and size validation. I also learned to connect the UI with the database and display success or failure messages.

Practical-13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Theory :

1. **Do PHP endpoints return valid JSON responses?**

   Answer: Yes, PHP endpoints return valid JSON responses for frontend communication.

2. **Are GET and POST requests handled properly?**

   Answer: Yes, GET and POST requests are handled correctly to retrieve and submit data.

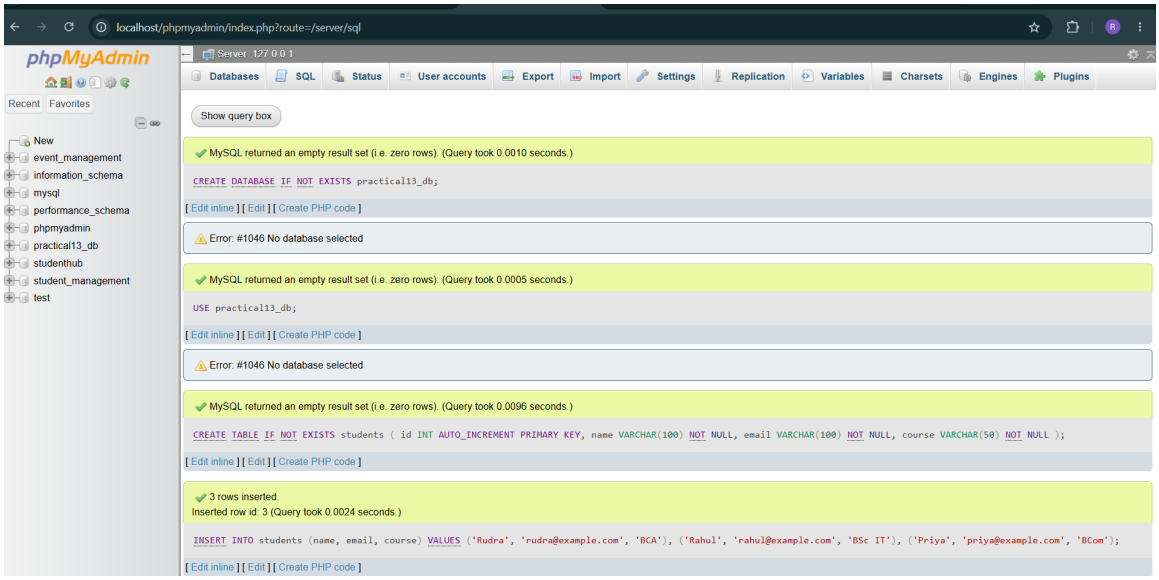
3. **Are loading, success, and error states shown on the frontend?**

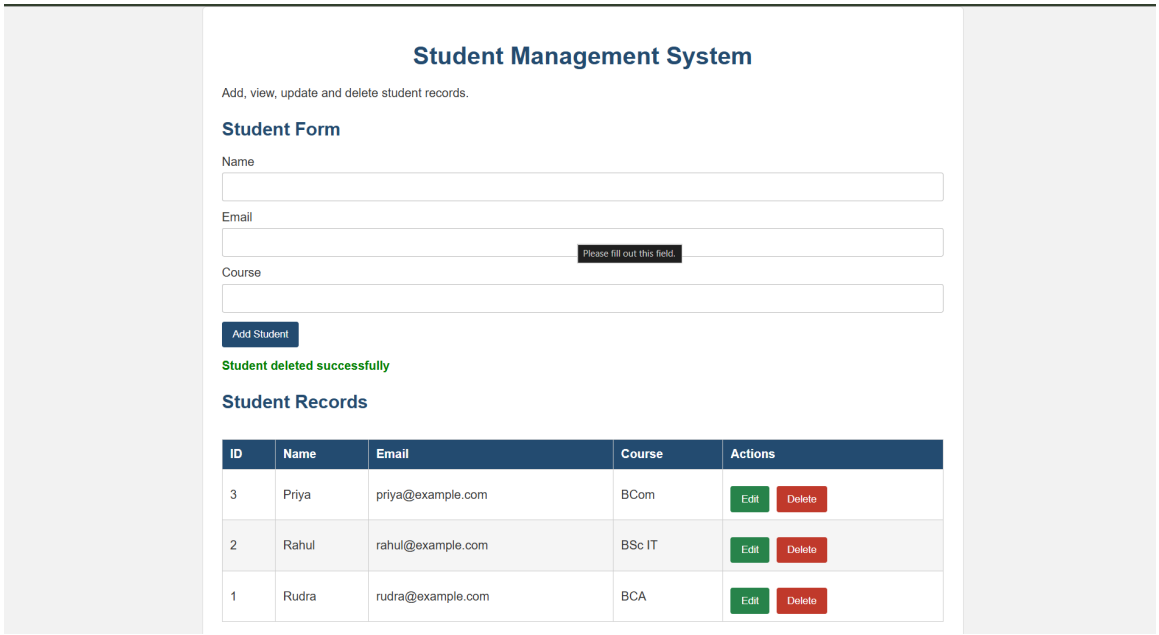
Answer: Yes, appropriate loading indicators and success or error messages are displayed.

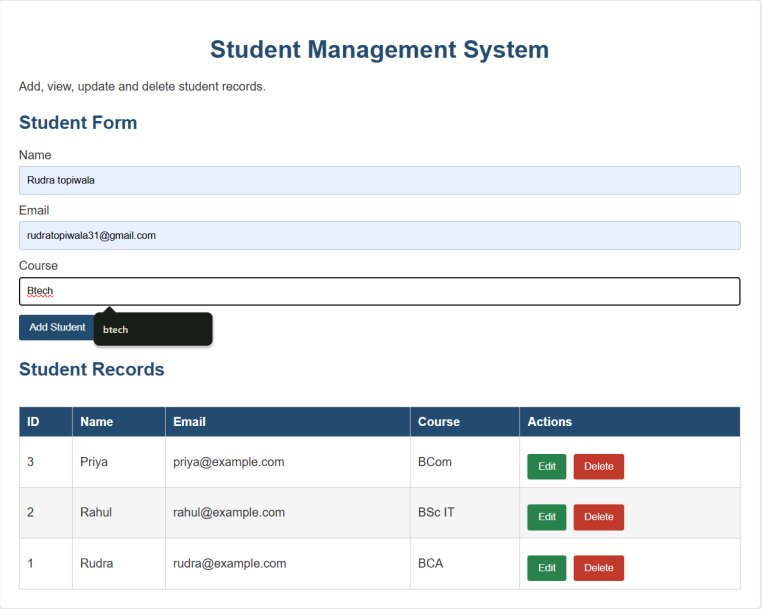
4. **Is input validation performed before database operations?**

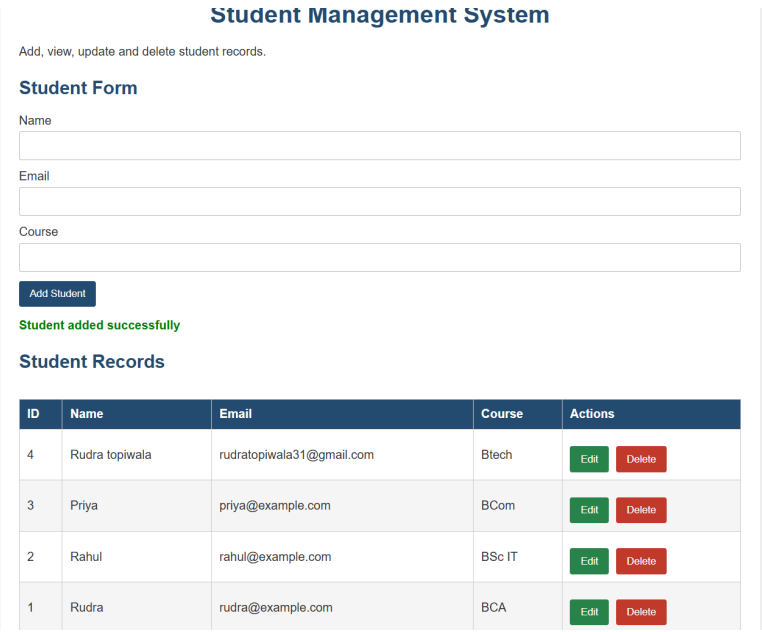
Answer: Yes, user inputs are validated before performing database operations.

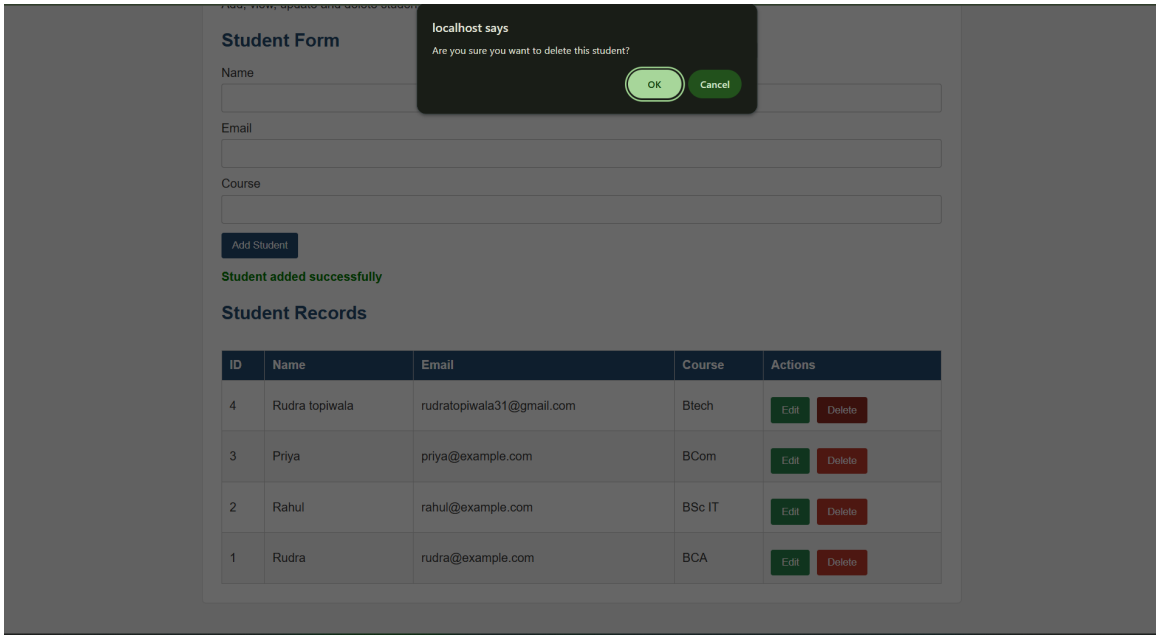
Result:


localhost/phpmyadmin/index.php?route=/server/sql
phpMyAdmin
Recent
Favorites
New
event_management
information_schema
mysql
performance_schema
phpmyadmin
practical13_db
studenthub
student_management
test
Server: 127.0.0.1
Databases
SQL
Status
User accounts
Export
Import
Settings
Replication
Variables
Charsets
Engines
Plugins
Show query box
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)
CREATE DATABASE IF NOT EXISTS practical13_db;
[Edit inline] [Edit] [Create PHP code]
Error: #1046 No database selected
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)
USE practical13_db;
[Edit inline] [Edit] [Create PHP code]
Error: #1046 No database selected
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0096 seconds.)
CREATE TABLE IF NOT EXISTS students (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, course VARCHAR(50) NOT NULL);
[Edit inline] [Edit] [Create PHP code]
3 rows inserted.
Inserted row id: 3 (Query took 0.0024 seconds.)
INSERT INTO students (name, email, course) VALUES ('Rudra', 'rudra@example.com', 'BCA'), ('Rahul', 'rahul@example.com', 'BSc IT'), ('Priya', 'priya@example.com', 'BCom');
[Edit inline] [Edit] [Create PHP code]


Student Management System
Add, view, update and delete student records.
Student Form
Name
Email
Please fill out this field.
Course
Add Student
Student deleted successfully
Student Records
ID
Name
Email
Course
Actions
3
Priya
priya@example.com
BCom
Edit
Delete
2
Rahul
rahul@example.com
BSc IT
Edit
Delete
1
Rudra
rudra@example.com
BCA
Edit
Delete


Student Management System
Add, view, update and delete student records.
Student Form
Name
Rudra topiwala
Email
rudratopiwala31@gmail.com
Course
Btech
Add Student
btech
Student Records
ID
Name
Email
Course
Actions
3
Priya
priya@example.com
BCom
Edit
Delete
2
Rahul
rahul@example.com
BSc IT
Edit
Delete
1
Rudra
rudra@example.com
BCA
Edit
Delete


Student Management System
Add, view, update and delete student records.
Student Form
Name
Email
Course
Add Student
Student added successfully
Student Records
ID
Name
Email
Course
Actions
4
Rudra topiwala
rudratopiwala31@gmail.com
Btech
Edit
Delete
3
Priya
priya@example.com
BCom
Edit
Delete
2
Rahul
rahul@example.com
BSc IT
Edit
Delete
1
Rudra
rudra@example.com
BCA
Edit
Delete



Git Hub Repository Link:

https://github.com/Dhanraj2408/StudentHub

Challenges Faced: I faced difficulties in creating PHP endpoints that return valid JSON responses. I struggled with using the Fetch API to perform CRUD operations without reloading the page. I also faced challenges in handling errors, validating inputs, and displaying appropriate messages.

Learning Outcome: I learned to create PHP JSON APIs and connect them with JavaScript using the Fetch API. I understood how to perform CRUD operations asynchronously without reloading the page. I also learned to validate user inputs and handle loading, success, and error messages.

Practical-14

Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Theory :

1. **Are dashboard values loaded dynamically from the database?**

   Answer: Yes, dashboard values are fetched dynamically from the database.

2. Is access restricted to admin users?

Answer: Yes, only authorized admin users can access the admin dashboard.

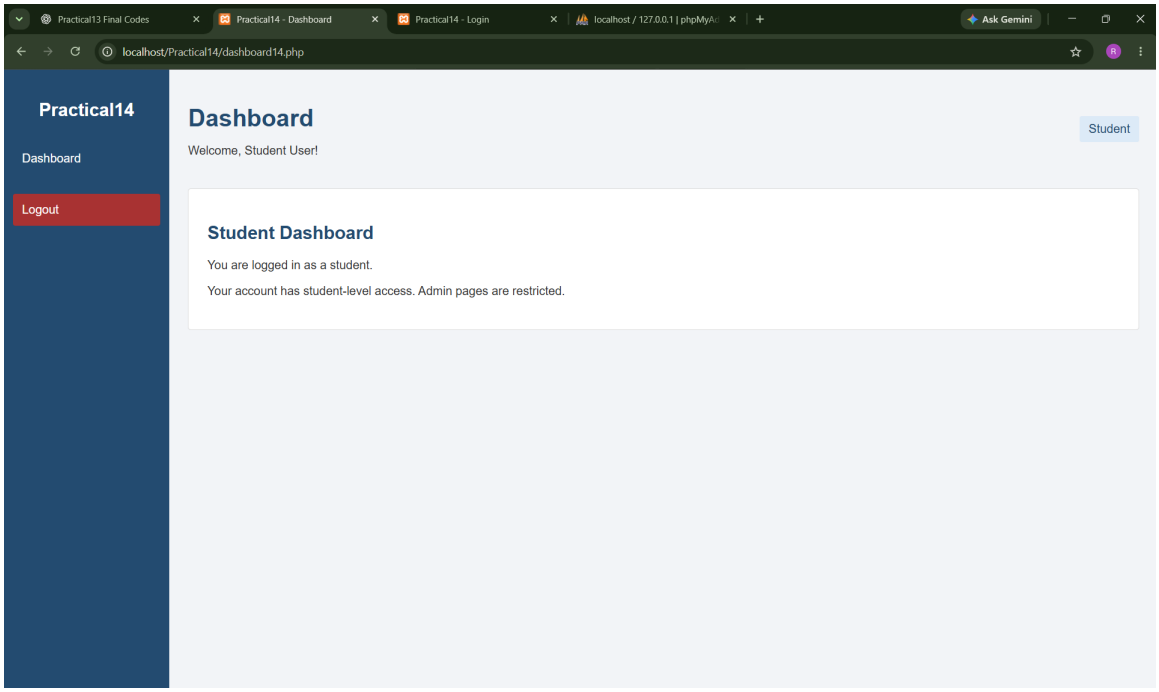
3. Are role-based menus working?

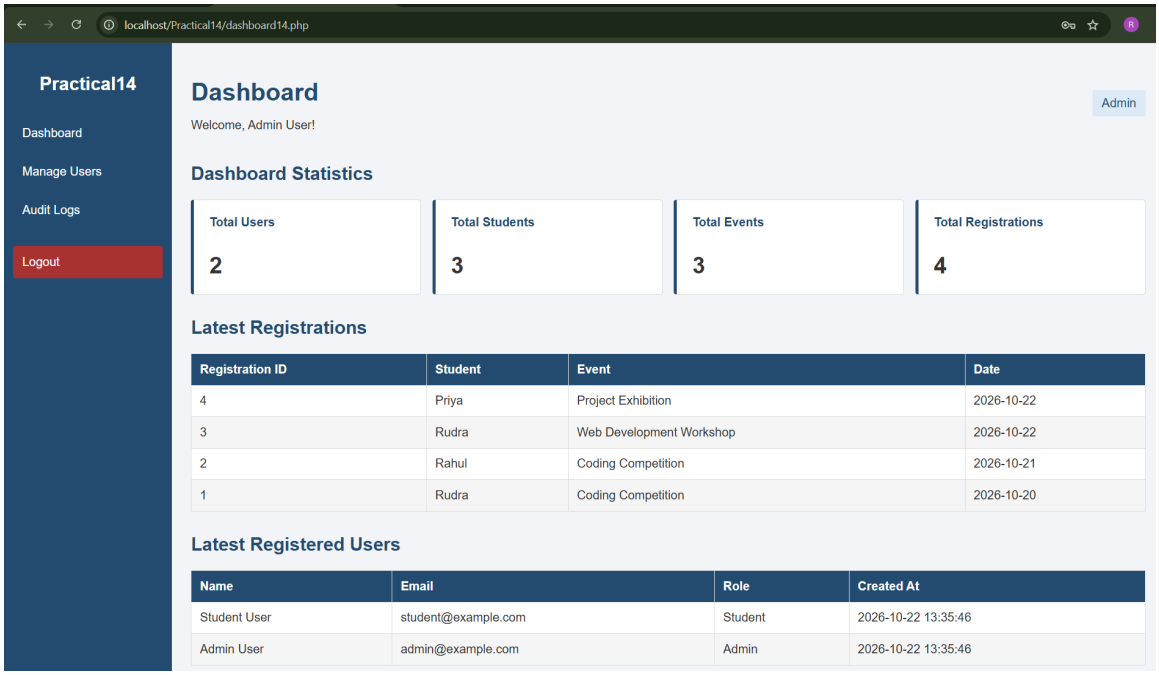
Answer: Yes, menus are displayed according to the user's role and permissions.

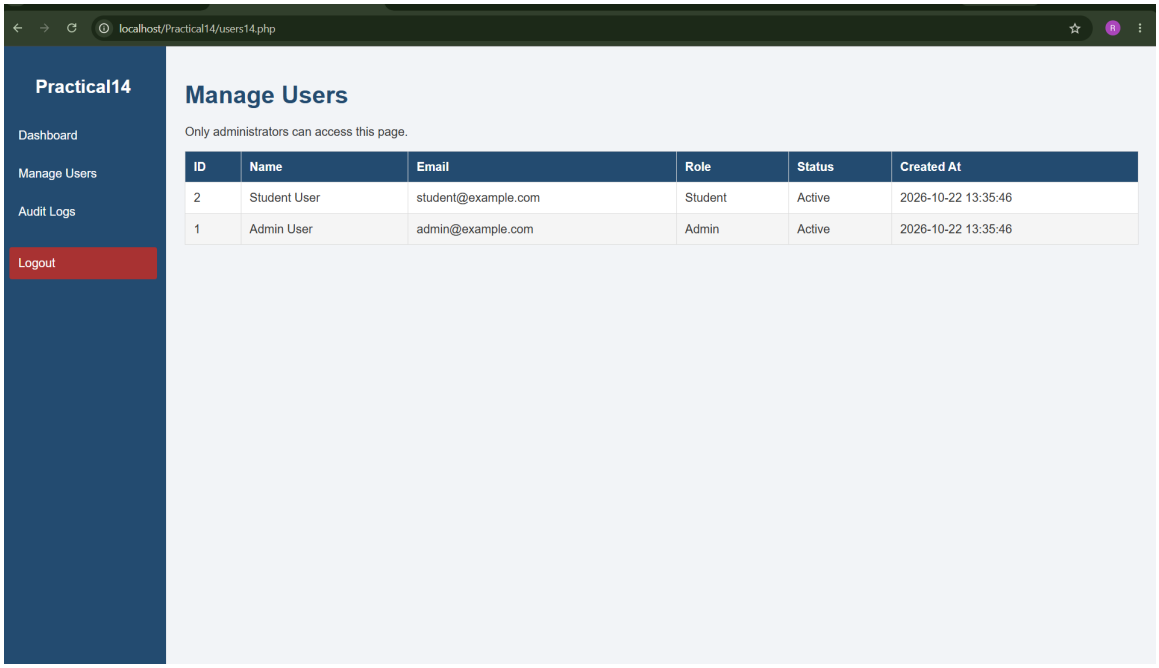
4. Are important admin actions recorded in audit_logs?

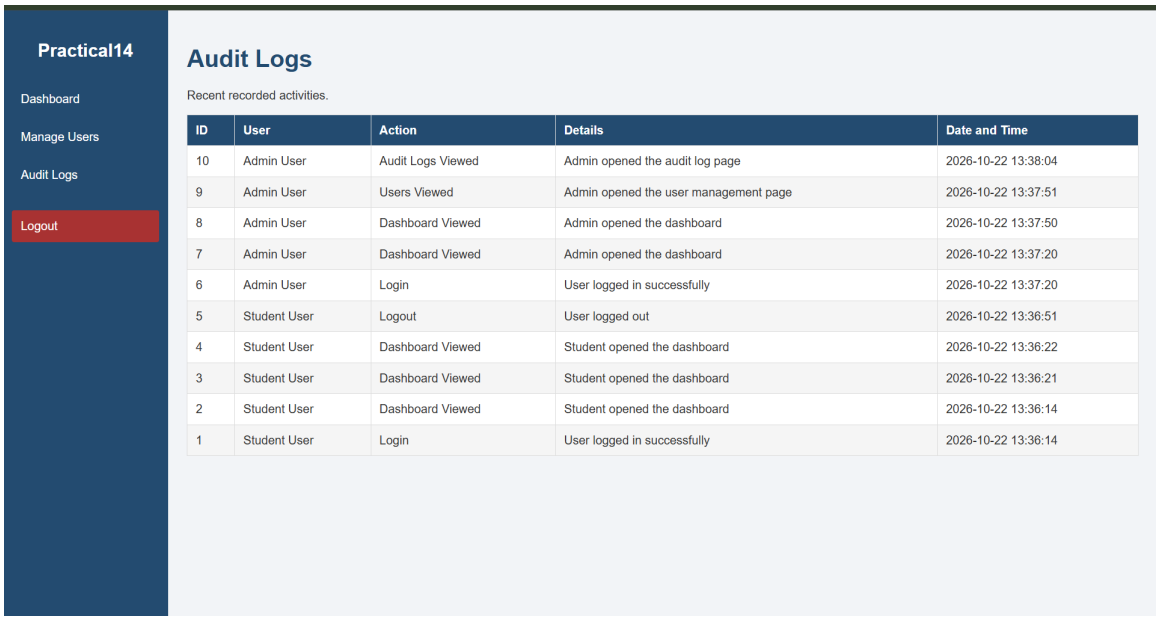
Answer: Yes, important admin actions are recorded in the audit_logs table for tracking and monitoring.

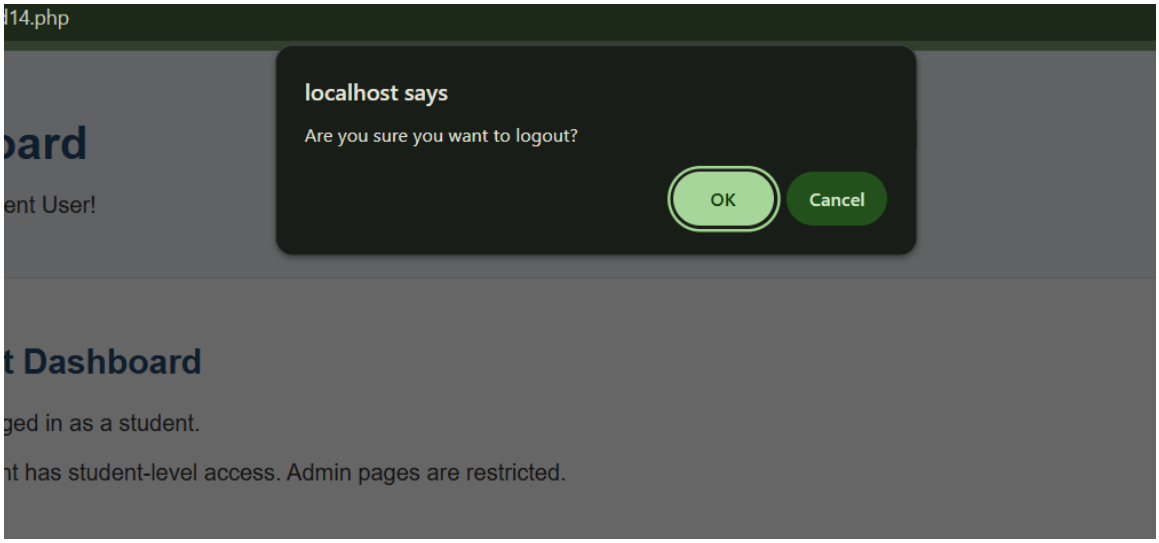
Result:

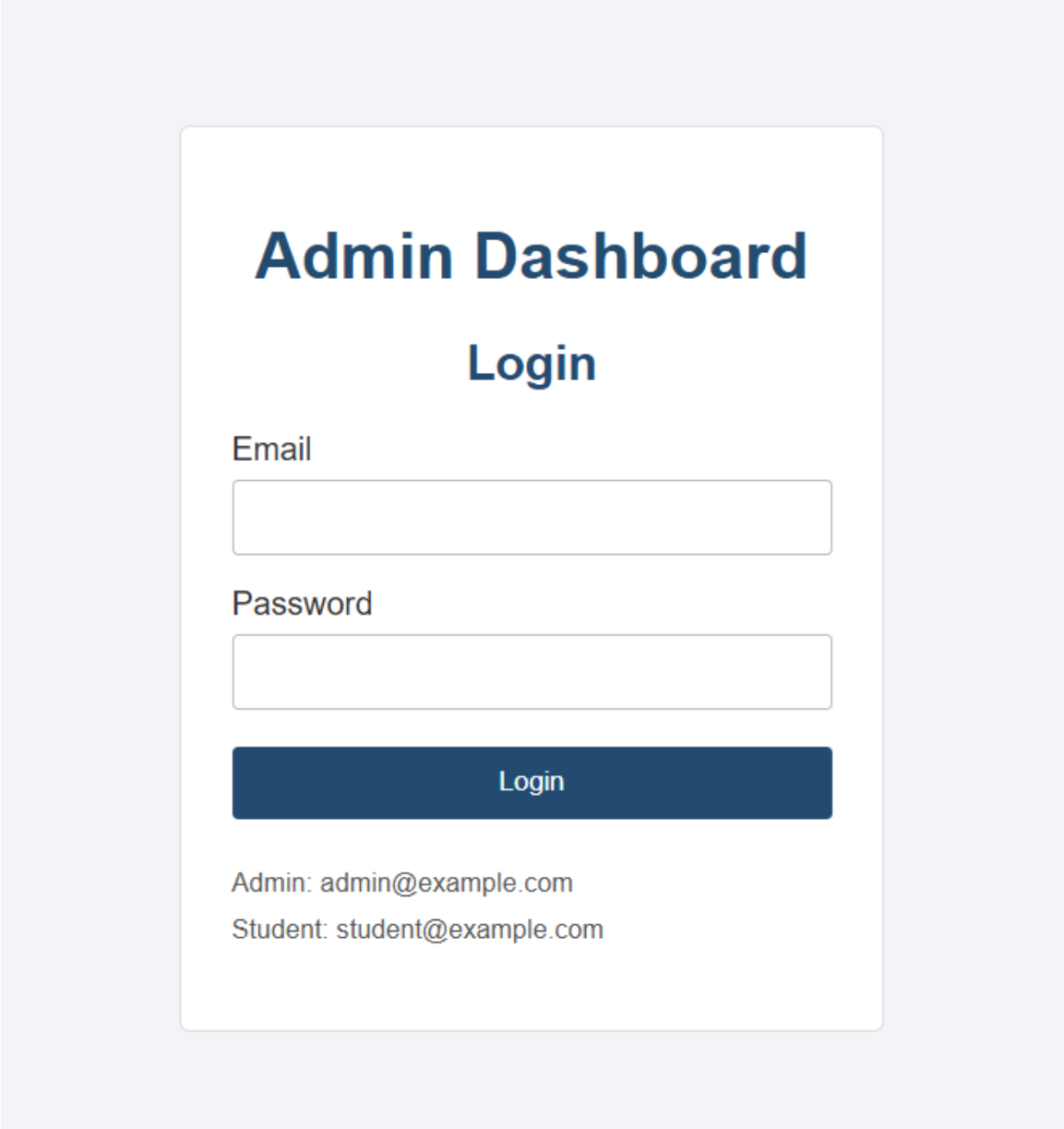

Practical13 Final Codes
Practical14 - Dashboard
Practical14 - Login
localhost / 127.0.0.1 | phpMyA
Ask Gemini
localhost/Practical14/dashboard14.php
Practical14
Dashboard
Logout
Dashboard
Welcome, Student User!
Student
Student Dashboard
You are logged in as a student.
Your account has student-level access. Admin pages are restricted.


localhost/Practical14/dashboard14.php
Practical14
Dashboard
Manage Users
Audit Logs
Logout
Dashboard
Welcome, Admin User!
Admin
Dashboard Statistics
Total Users
2
Total Students
3
Total Events
3
Total Registrations
4
Latest Registrations
Registration ID
Student
Event
Date
4
Priya
Project Exhibition
2026-10-22
3
Rudra
Web Development Workshop
2026-10-22
2
Rahul
Coding Competition
2026-10-21
1
Rudra
Coding Competition
2026-10-20
Latest Registered Users
Name
Email
Role
Created At
Student User
student@example.com
Student
2026-10-22 13:35:46
Admin User
admin@example.com
Admin
2026-10-22 13:35:46


localhost/Practical14/users14.php
Practical14
Dashboard
Manage Users
Audit Logs
Logout
Manage Users
Only administrators can access this page.
ID
Name
Email
Role
Status
Created At
2
Student User
student@example.com
Student
Active
2026-10-22 13:35:46
1
Admin User
admin@example.com
Admin
Active
2026-10-22 13:35:46


Practical14
Dashboard
Manage Users
Audit Logs
Logout
Audit Logs
Recent recorded activities.
ID
User
Action
Details
Date and Time
10
Admin User
Audit Logs Viewed
Admin opened the audit log page
2026-10-22 13:38:04
9
Admin User
Users Viewed
Admin opened the user management page
2026-10-22 13:37:51
8
Admin User
Dashboard Viewed
Admin opened the dashboard
2026-10-22 13:37:50
7
Admin User
Dashboard Viewed
Admin opened the dashboard
2026-10-22 13:37:20
6
Admin User
Login
User logged in successfully
2026-10-22 13:37:20
5
Student User
Logout
User logged out
2026-10-22 13:36:51
4
Student User
Dashboard Viewed
Student opened the dashboard
2026-10-22 13:36:22
3
Student User
Dashboard Viewed
Student opened the dashboard
2026-10-22 13:36:21
2
Student User
Dashboard Viewed
Student opened the dashboard
2026-10-22 13:36:14
1
Student User
Login
User logged in successfully
2026-10-22 13:36:14


14.php
localhost says
Are you sure you want to logout?
OK
Cancel
ard
ent User!
t Dashboard
ged in as a student.
nt has student-level access. Admin pages are restricted.



Git Hub Repository Link:

https://github.com/Dhanraj2408/StudentHub

Challenges Faced: I faced difficulties in fetching dashboard statistics dynamically from the database. I struggled with implementing role-based access control and displaying menus according to user roles. I also faced challenges in recording important admin activities in the audit_logs table.

Learning Outcome: I learned to create an admin dashboard with dynamic statistics and latest registration details. I understood how to restrict dashboard access to admins and display role-based menus. I also learned how to maintain audit logs to track important administrative activities.